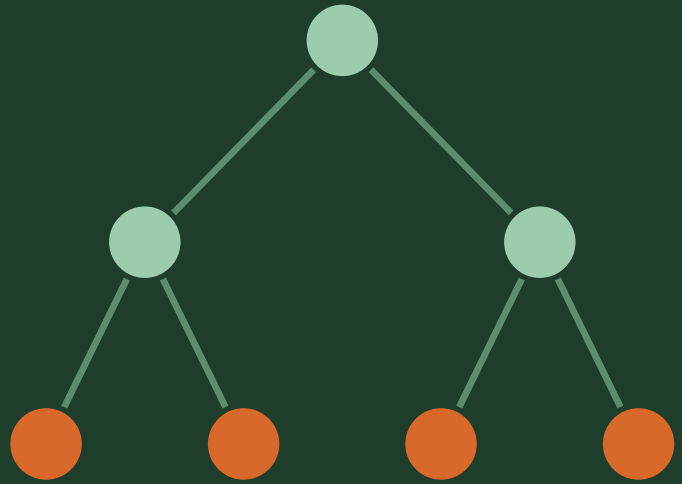
Деревья

Структура данных, на которой держатся папки, веб-страницы и std::map



Структуры данных · C++

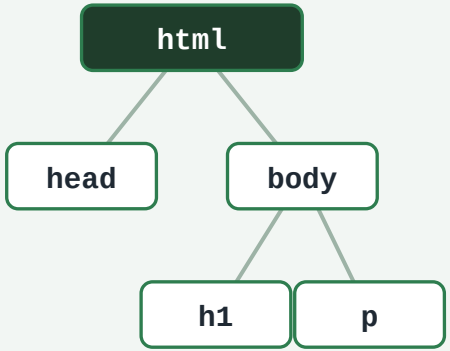
Вы пользуетесь деревьями каждый день

Папки на компьютере



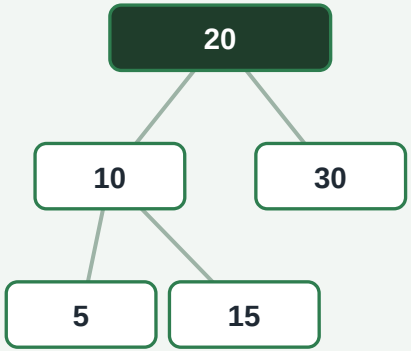
Каждая папка лежит ровно в одной родительской папке.

Веб-страница



Браузер хранит страницу как дерево тегов (DOM).

std::map и std::set



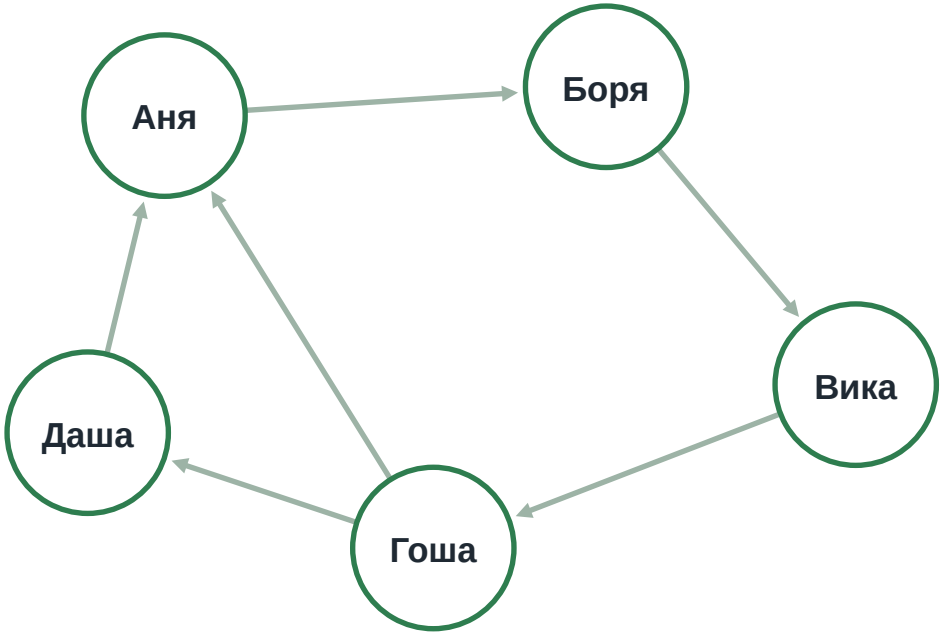
Внутри — сбалансированное дерево поиска.

Часть 1

Граф и дерево

Откуда берутся деревья и чем они отличаются от обычного графа

Граф — это точки и связи между ними



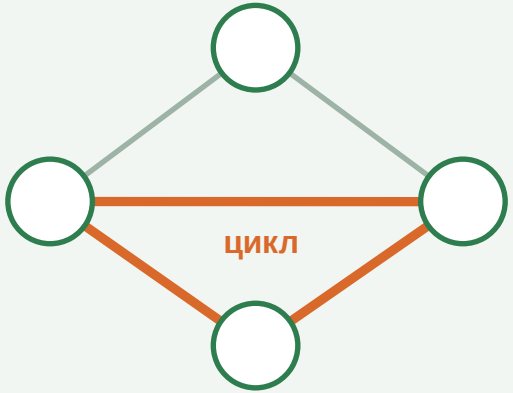
Вершины — объекты: люди, города, страницы.

Рёбра — связи между ними. Стрелка «Аня → Боря» значит «Аня подписана на Борю».

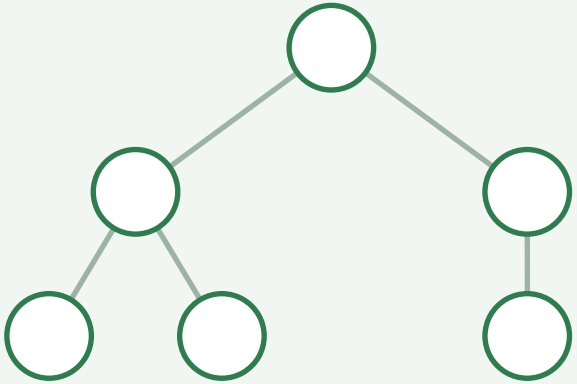
В графе бывают **циклы**: Аня → Боря → Вика → Гоша → Аня.

Дерево — это граф без циклов

Граф с циклом

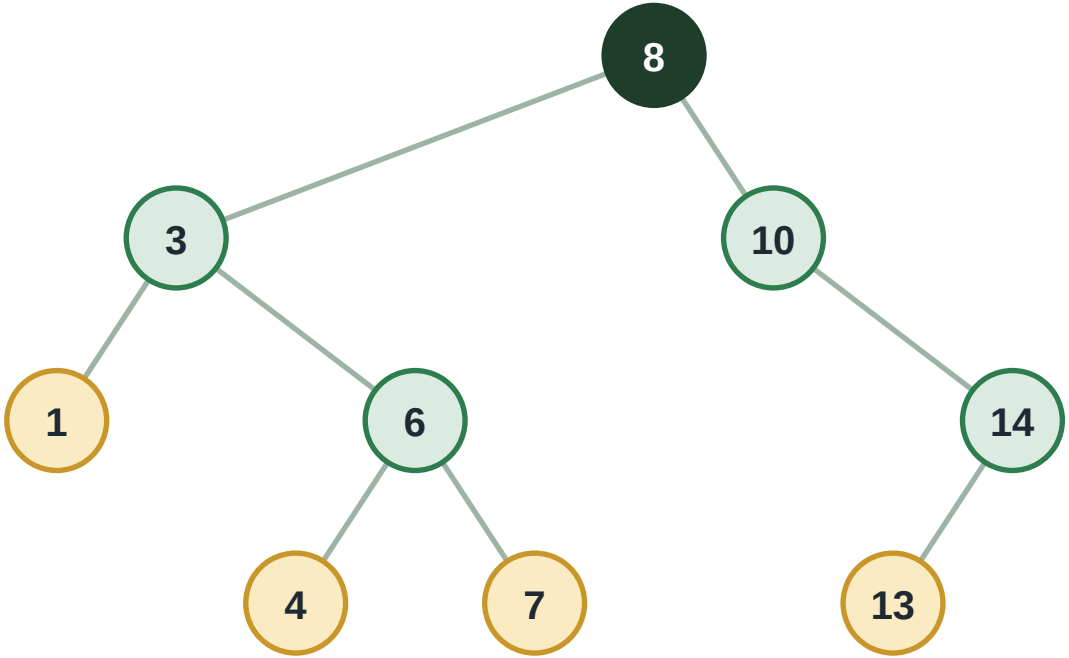


Дерево



Три правила дерева: все вершины связаны; циклов нет; если вершин **N**, то рёбер ровно **N − 1**. Поэтому между любыми двумя вершинами ровно один путь.

Из чего состоит дерево



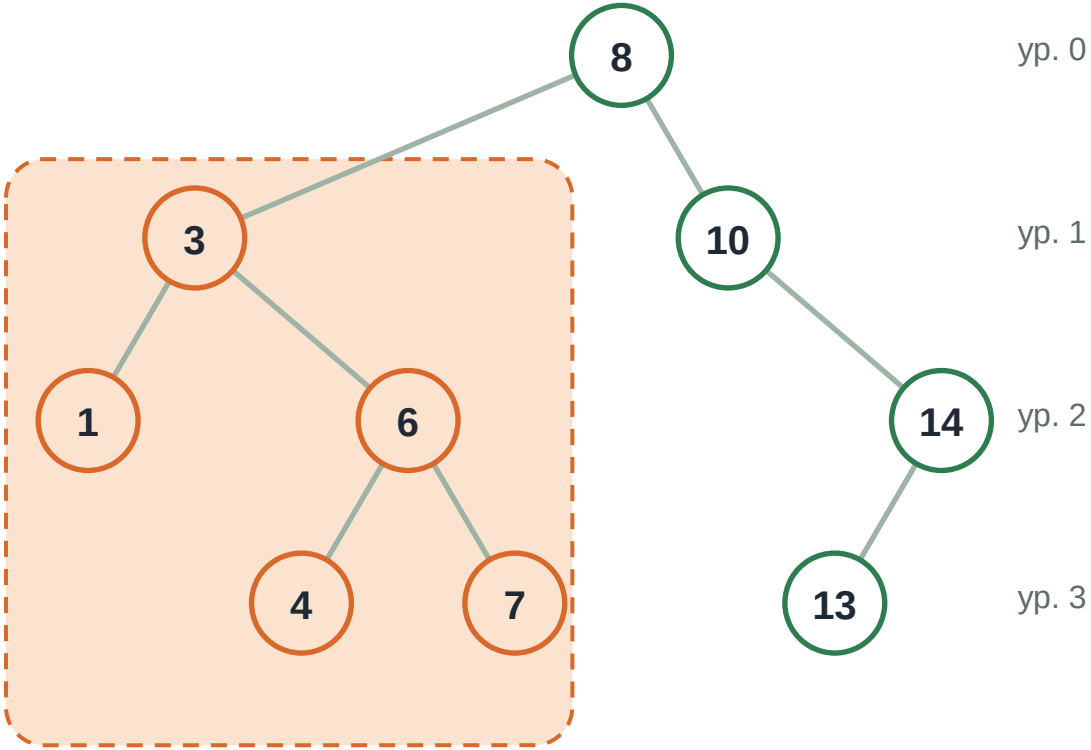
Корень
Самый верхний узел. Здесь это 8.

Внутренние узлы
Есть хотя бы один ребёнок: 3, 10, 6, 14.

Листья
Детей нет: 1, 4, 7, 13.

Узел 3 — **родитель** для 1 и 6, а они — его **дети**.

Поддерево и высота



Поддерево — узел вместе со всеми его потомками. Оранжевое — поддерево с корнем 3. Это тоже дерево!

Высота — сколько рёбер от узла до самого дальнего листа под ним.

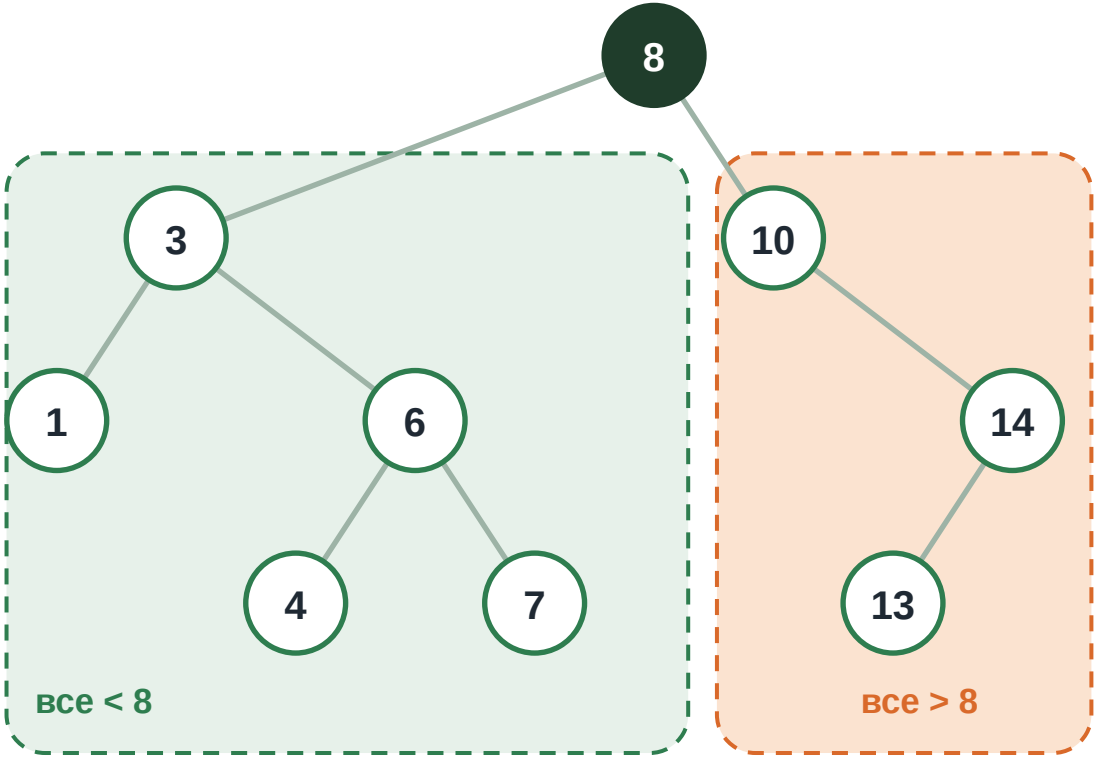
Высота всего дерева = 3 (8 → 3 → 6 → 4).

Часть 2

Двоичное дерево поиска (BST)

Binary Search Tree: одно простое правило, которое делает поиск очень быстрым

Правило BST: слева меньше, справа больше



BST (Binary Search Tree) — двоичное дерево поиска.

Двоичное: у каждого узла не больше двух детей — левый и правый.

Поиска: в левом поддереве все ключи меньше узла, в правом — больше.

Правило работает для **каждого** узла: у 3 слева 1, справа 6, 4, 7.

Каждое сравнение отбрасывает половину дерева

Ищем одно число среди 1 000 000 чисел:

Список или массив

1 000 000

проверок в худшем случае: смотрим каждый элемент по очереди

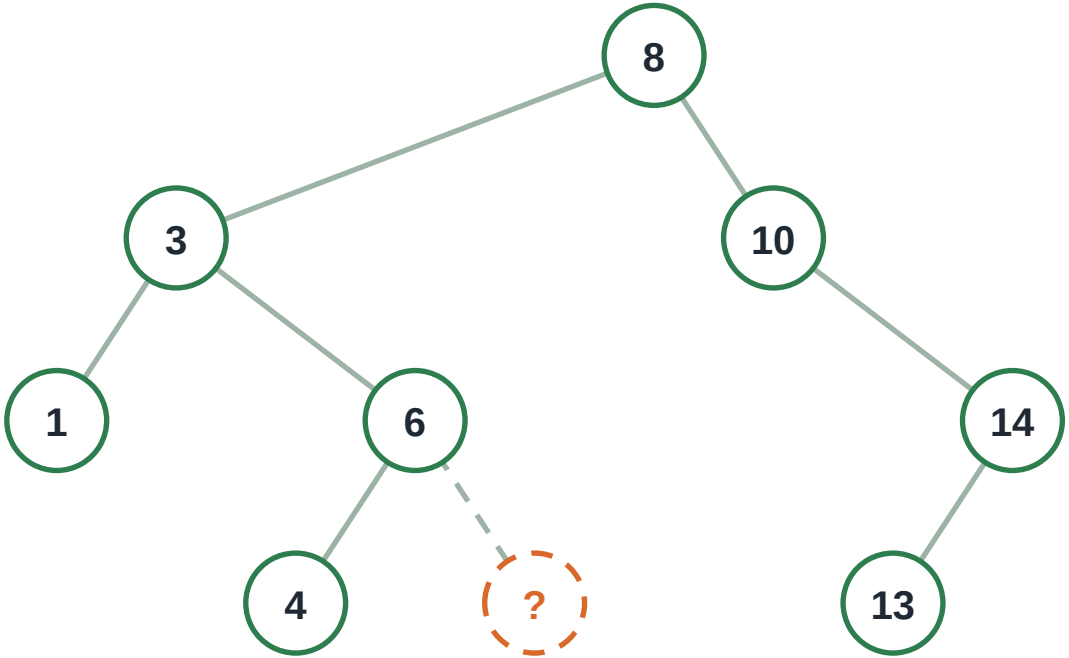
Сбалансированное дерево

≈ 20

проверок: 1 000 000 → 500 000 → 250 000 → … → 1

Это тот же приём, что в игре «угадай число от 1 до 100»: спрашиваешь «больше 50?» и сразу отбрасываешь половину.

Вставка в BST: спускаемся до пустого места



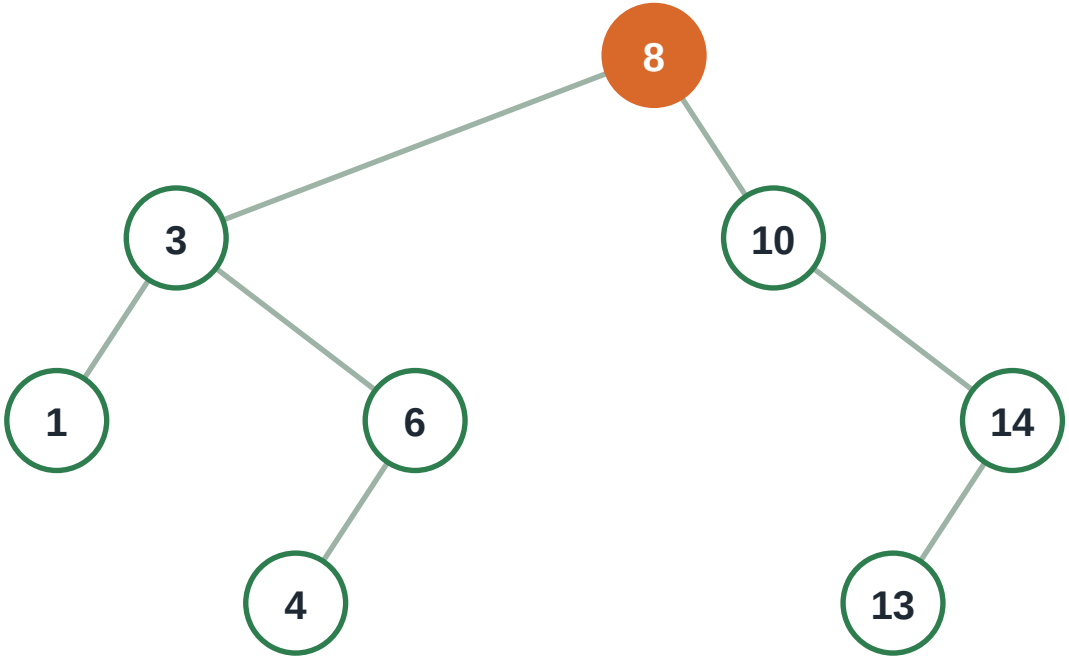
Задача: вставить **7**.

Начинаем с корня. На каждом узле сравниваем:

- **меньше → идём налево**
- **больше → идём направо**

Дошли до пустого места — ставим новый узел туда.

Вставка 7 · шаг 1 из 4

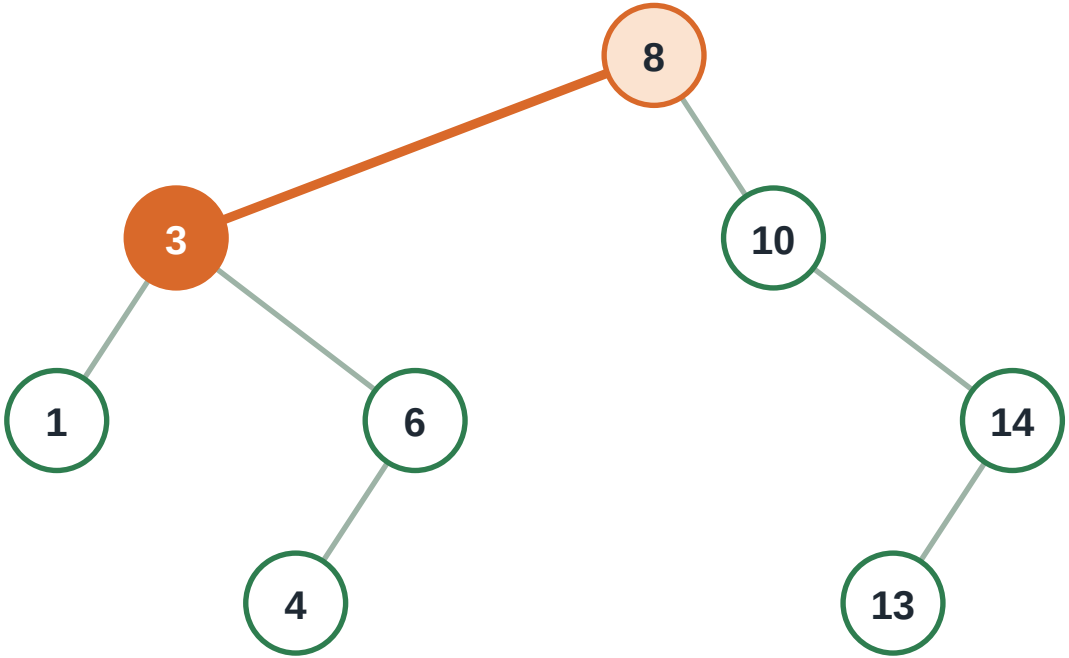


7 < 8

Меньше. Идём в левое поддерево.

Путь: 8

Вставка 7 · шаг 2 из 4

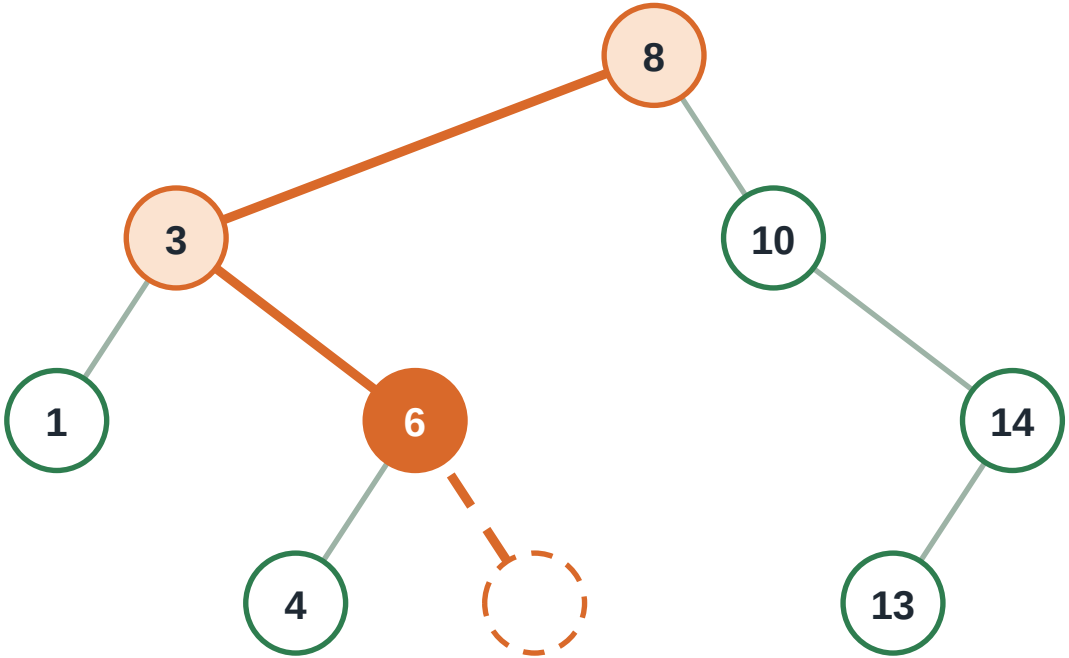


7 > 3

Больше. Идём в правое поддерево.

Путь: 8 → 3

Вставка 7 · шаг 3 из 4

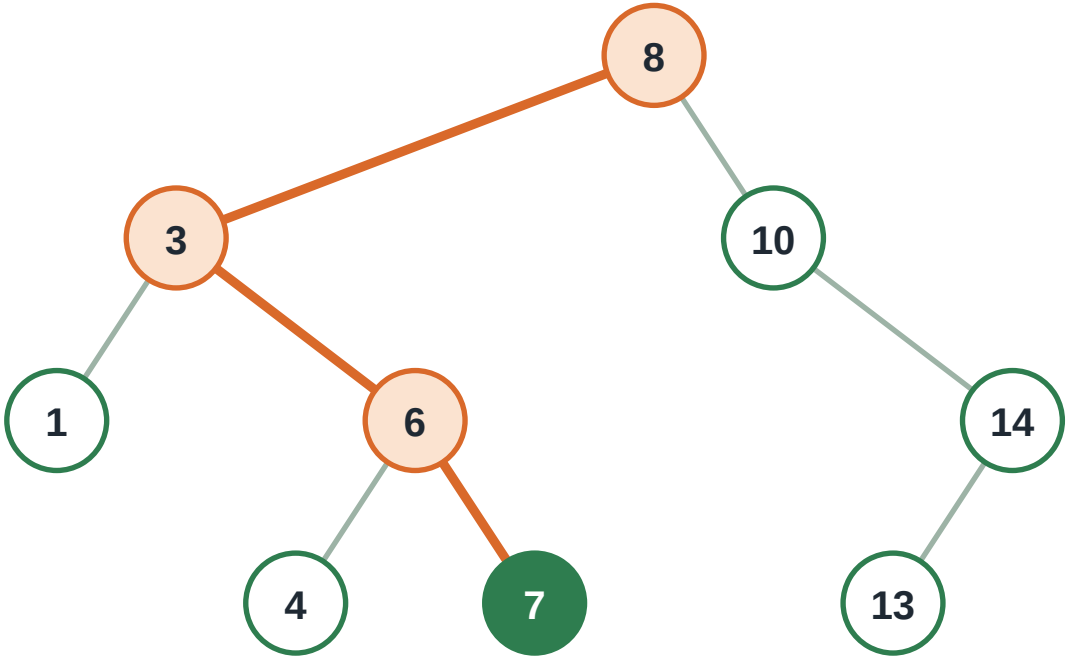


7 > 6

Больше. Справа у 6 пусто: это место для нового узла.

Путь: 8 → 3 → 6

Вставка 7 · шаг 4 из 4

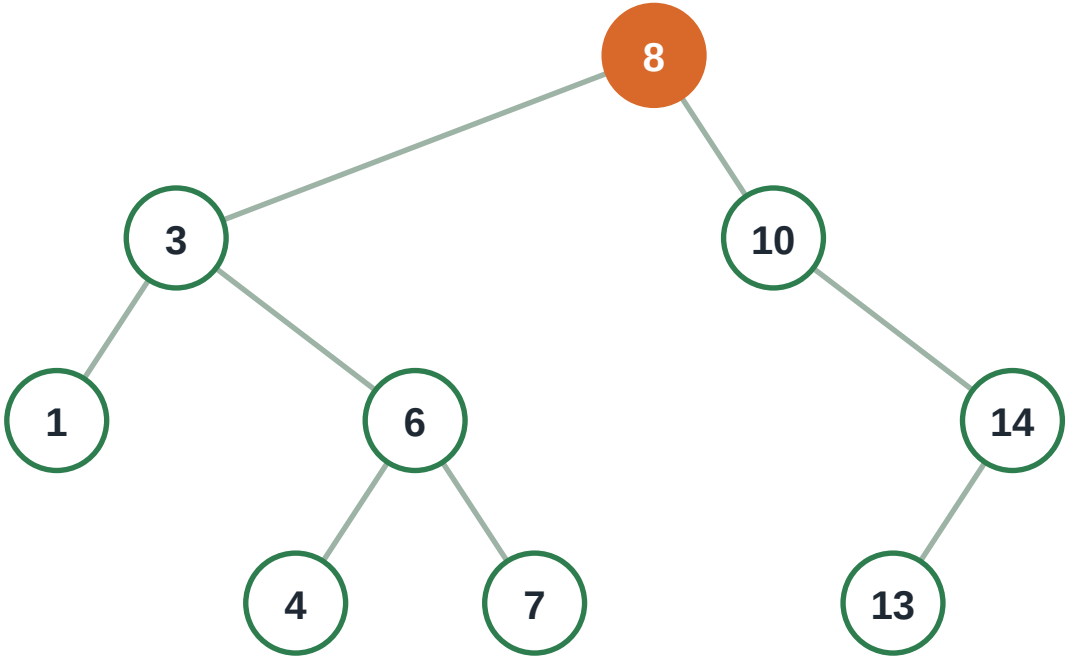


Готово!

Узел 7 стал правым ребёнком 6. Правило «слева меньше, справа больше» не нарушено.

Путь: 8 → 3 → 6 → 7

Поиск 4 · шаг 1 из 4

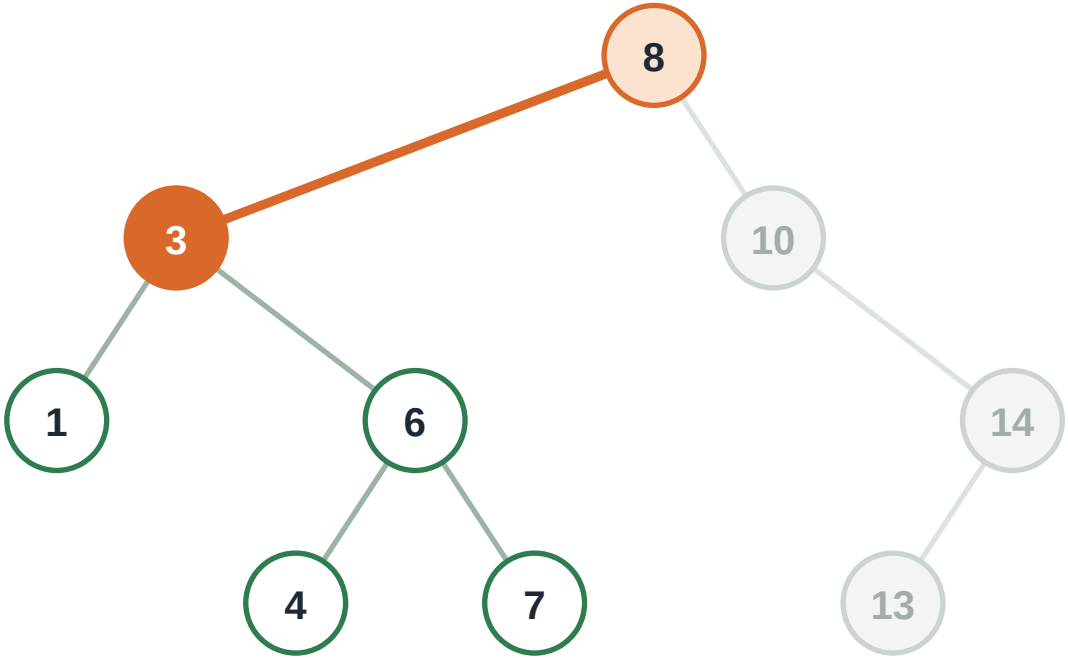


4 < 8

Меньше. Всё правое поддерево (10, 13, 14) можно не проверять.

Путь: 8

Поиск 4 · шаг 2 из 4

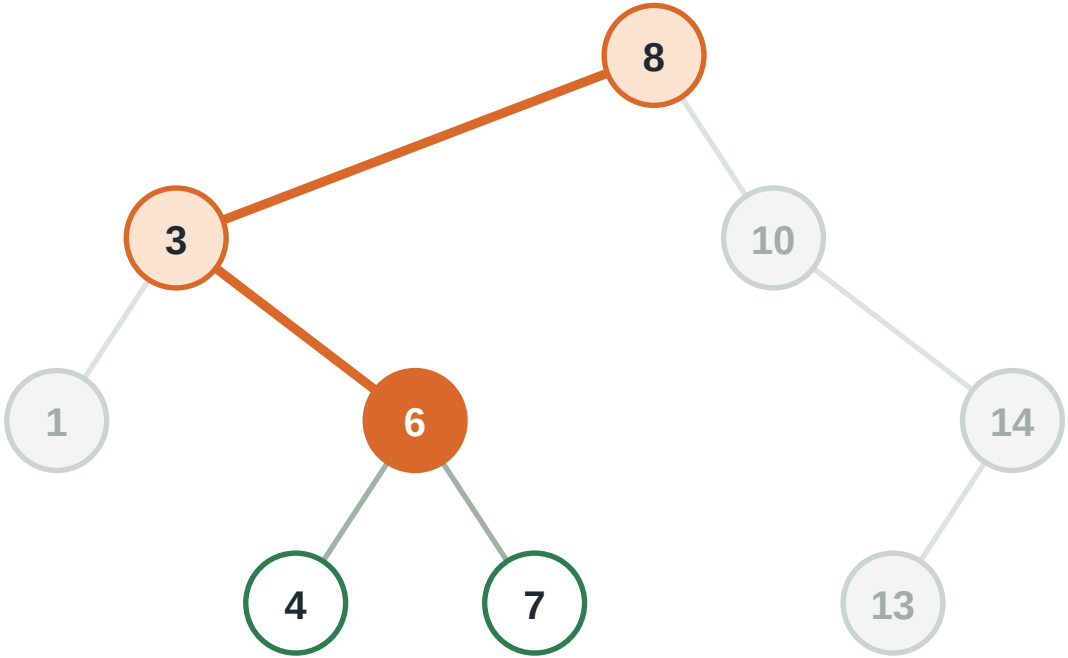


4 > 3

Больше. Отбрасываем узел 1.

Путь: 8 → 3

Поиск 4 · шаг 3 из 4

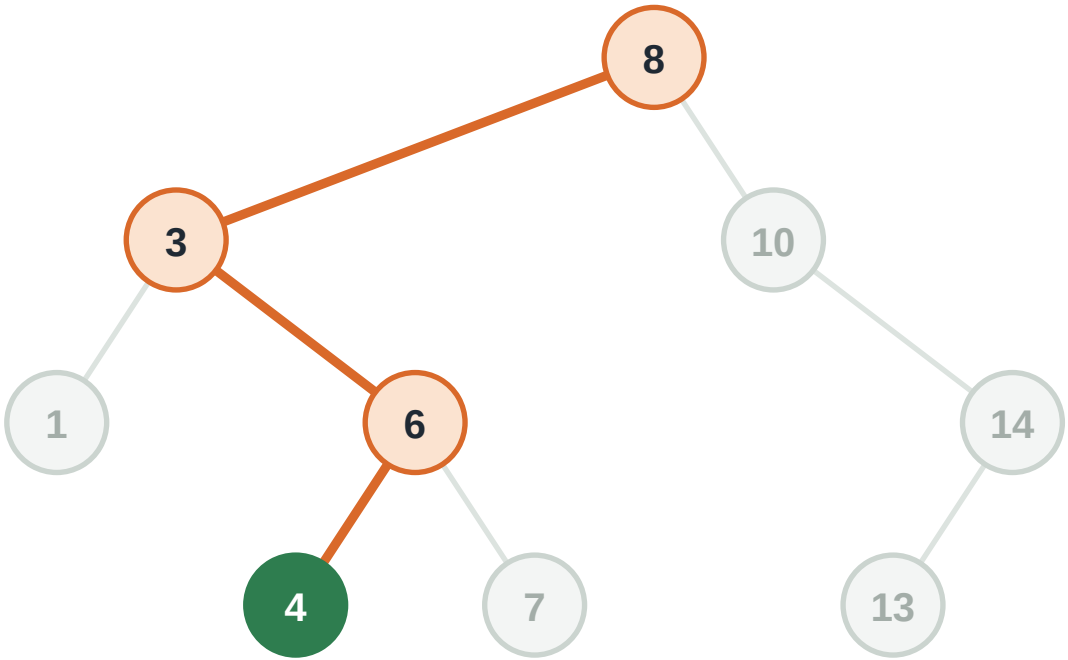


4 < 6

Меньше. Отбрасываем 7.

Путь: 8 → 3 → 6

Поиск 4 · шаг 4 из 4

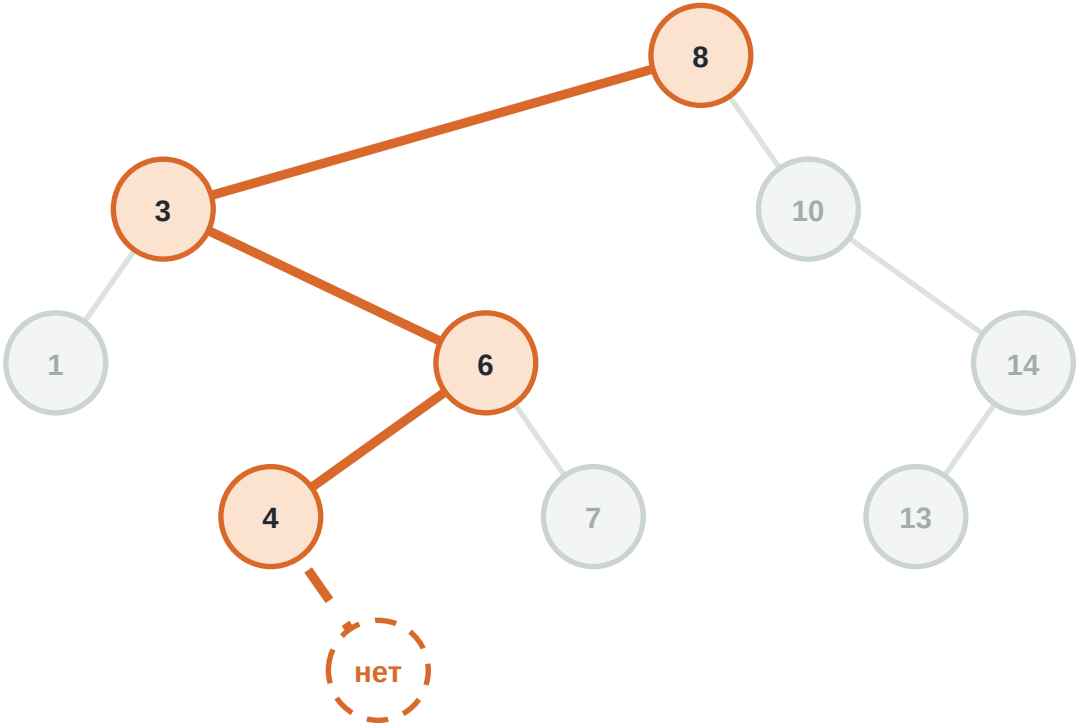


Нашли!

4 найдено за 4 сравнения. Из 9 узлов проверили только 4.

Путь: 8 → 3 → 6 → 4

А если числа нет? Ищем 5



Путь: 8 → 3 → 6 → 4.

5 > 4, идём направо, а там пусто.

Если бы 5 было в дереве, оно стояло бы именно здесь. Значит, его нет.

В коде это значит: функция вернёт `nullptr`.

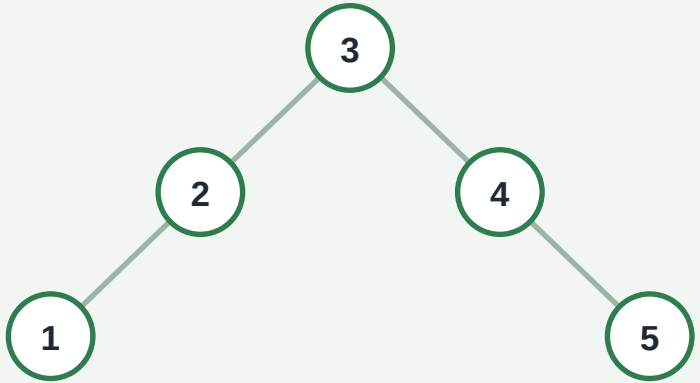
Одни и те же числа, а деревья разные

Вставляем 1, 2, 3, 4, 5



Высота 4. Это не дерево, а «палка»: поиск снова идёт по одному элементу, как в списке.

Вставляем 3, 2, 4, 1, 5



Высота 2. Каждое сравнение отбрасывает половину.

АВЛ-дерево само следит за балансом



Вставили 1, 2, 3 — начала расти «палка». Дерево замечает перекос и поворачивается: 2 поднимается наверх.

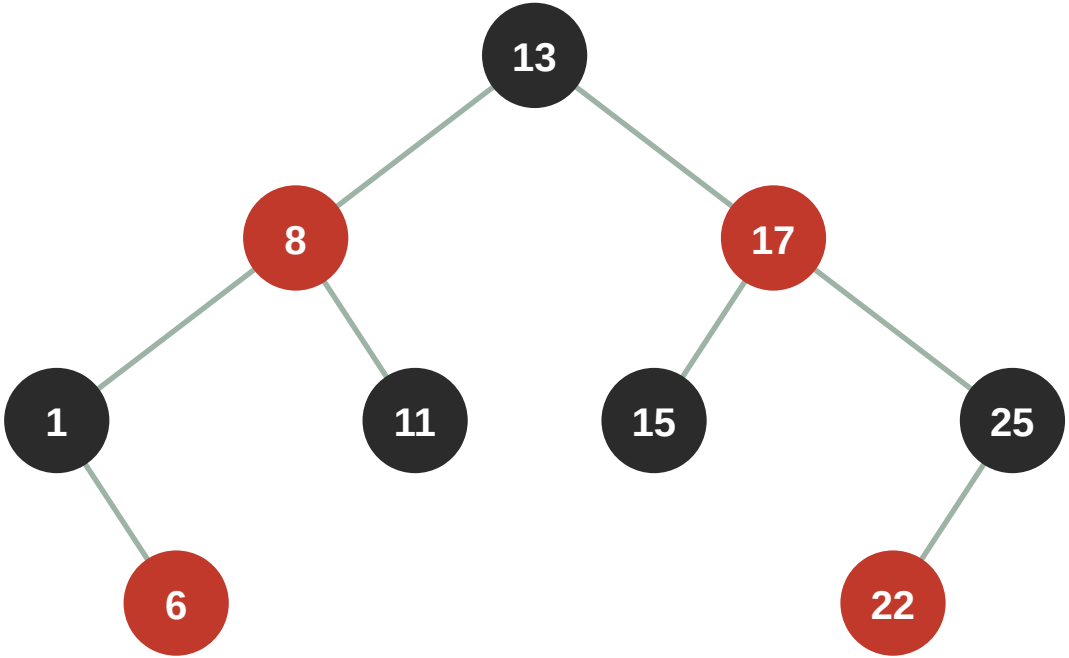
Правило АВЛ:

у каждого узла высоты левого и правого поддерева отличаются **не больше чем на 1**.

После каждой вставки и удаления дерево проверяет правило и при нарушении делает поворот.

Итог: высота всегда около $\log_2 N$.

Красно-чёрное дерево живёт внутри std::map



Для любопытных

Каждый узел покрашен в красный или чёрный. Главные правила:

- корень чёрный;
- у красного узла нет красных детей;
- на любом пути вниз одинаково чёрных узлов.

Это держит дерево почти сбалансированным. std::set и std::map в C++ обычно устроены именно так.

Часть 3

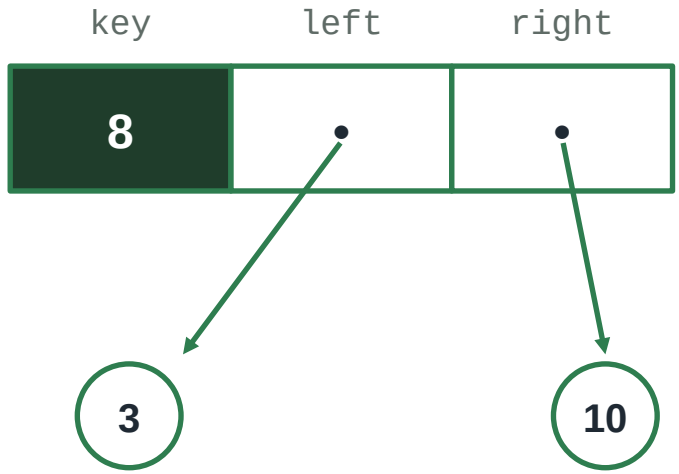
Пишем дерево на C++

Узел, вставка, поиск, обход и удаление

Узел хранит ключ и два указателя

```
struct Node {
    int key;
    Node* left = nullptr;
    Node* right = nullptr;

    Node(int k) : key(k) {}
};
```



`nullptr` в left или right значит «ребёнка нет». У листа оба указателя nullptr. Конструктор **`Node(int k)`** сразу кладёт ключ.

Вставка: спускаемся влево или вправо

```
Node* insert(Node* root, int key) {
    if (root == nullptr)
        return new Node(key);
    if (key < root->key)
        root->left = insert(root->left, key);
    else if (key > root->key)
        root->right = insert(root->right, key);
    return root;
}
```

1. **Пусто?**
   Нашли место — создаём узел через new и возвращаем его.
2. **Меньше или больше?**
   Вставляем в левое или правое поддерево и цепляем результат обратно.
3. **Равно?**
   Дубликаты не храним: просто возвращаем root.

Вызов: `root = insert(root, 7);` Присваивание нужно, потому что у пустого дерева корень появляется только при первой вставке.

Поиск повторяет путь вставки

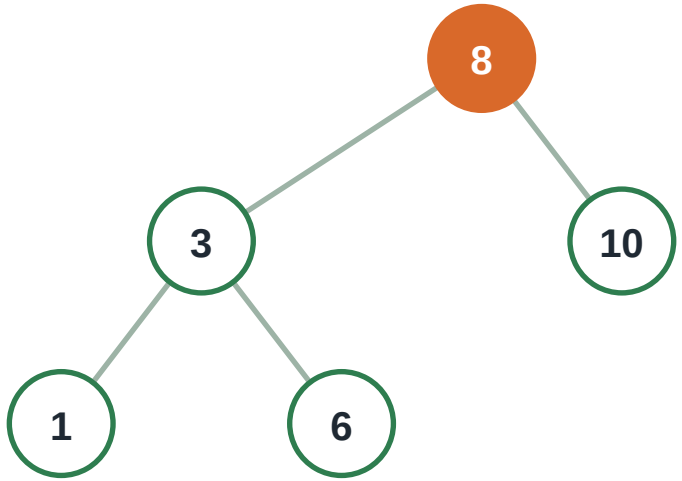
```
Node* find(Node* root, int key) {
    if (root == nullptr || root->key == key)
        return root;
    if (key < root->key)
        return find(root->left, key);
    return find(root->right, key);
}
```

1. **Стоп-условие**
   Пустое место — ключа нет (вернём nullptr). Совпал ключ — вернём этот узел.
2. **Шаг**
   Меньше — ищем слева, больше — справа. Вторую половину не трогаем.

Проверка: `if (find(root, 4) != nullptr) { … }`

Обход BST · шаг 1 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Начинаем с корня 8. Сначала обходим всё левое поддерево.

Стек вызовов

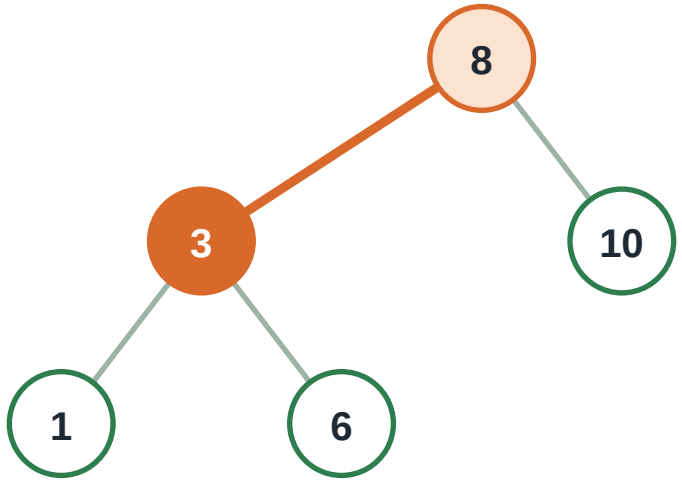
`traverse(8)`

Вывод

—

Обход BST · шаг 2 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Пришли в 3. Снова сначала налево.

Стек вызовов

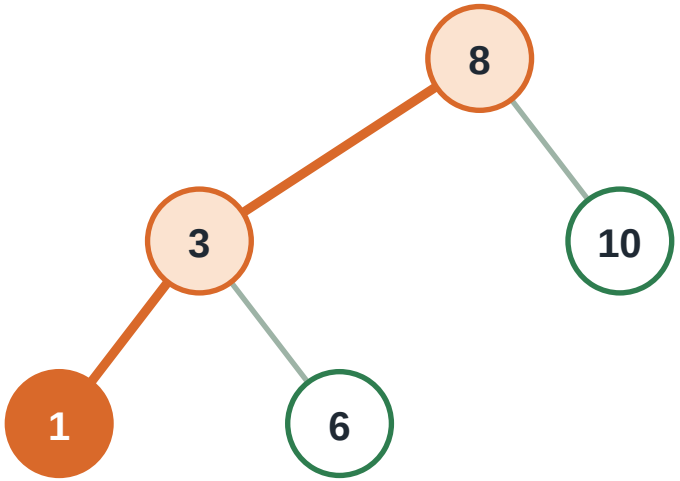
`traverse(8)` → `traverse(3)`

Вывод

—

Обход BST · шаг 3 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Слева у 1 пусто: traverse(nullptr) сразу сделал return. Печатаем 1.

Стек вызовов

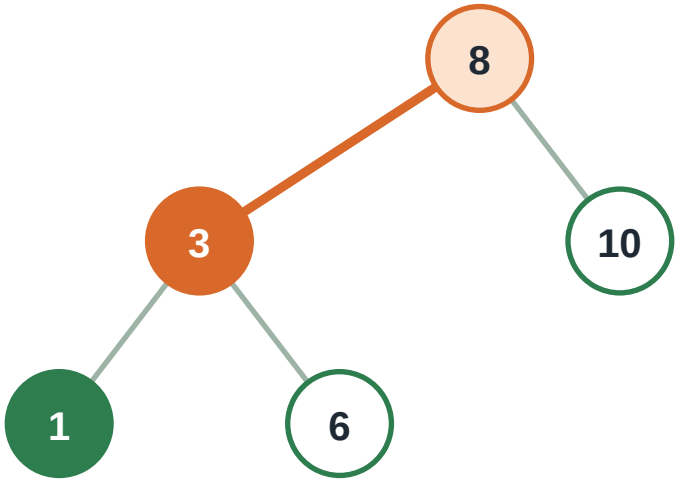
`traverse(8)` → `traverse(3)` → `traverse(1)`

Вывод

`1`

Обход BST · шаг 4 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Вызов для 1 закончился, вернулись в 3. Левое поддерево готово, печатаем 3.

Стек вызовов

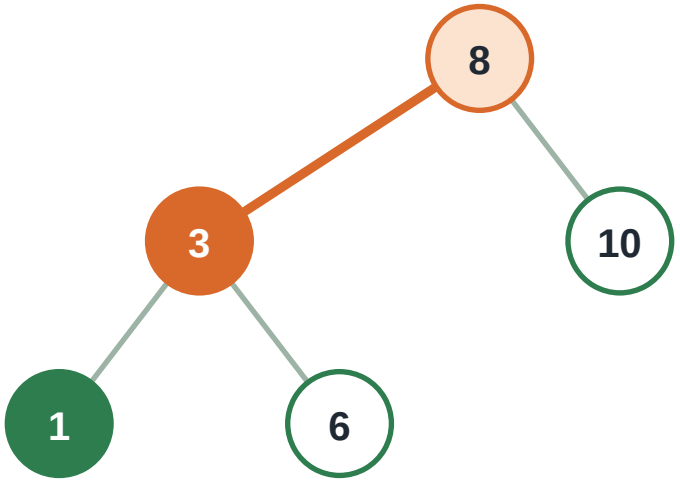
`traverse(8) → traverse(3)`

Вывод

`1 3`

Обход BST · шаг 5 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Теперь правое поддерево узла 3.

Стек вызовов

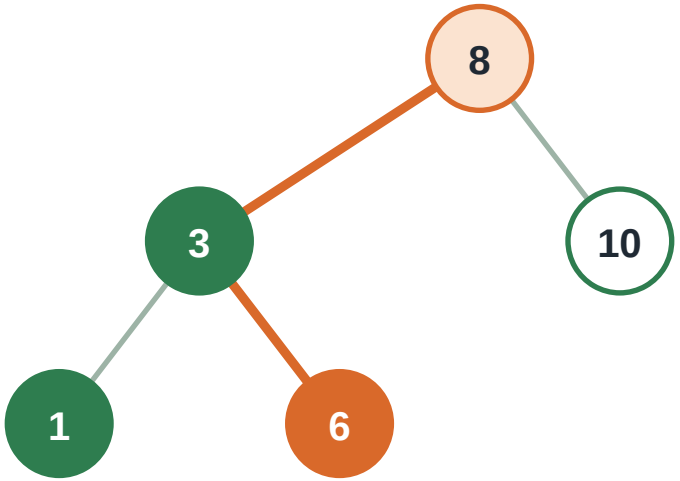
`traverse(8)` → `traverse(3)`

Вывод

`1 3`

Обход BST · шаг 6 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



6 — лист: слева пусто, печатаем 6, справа тоже пусто.

Стек вызовов

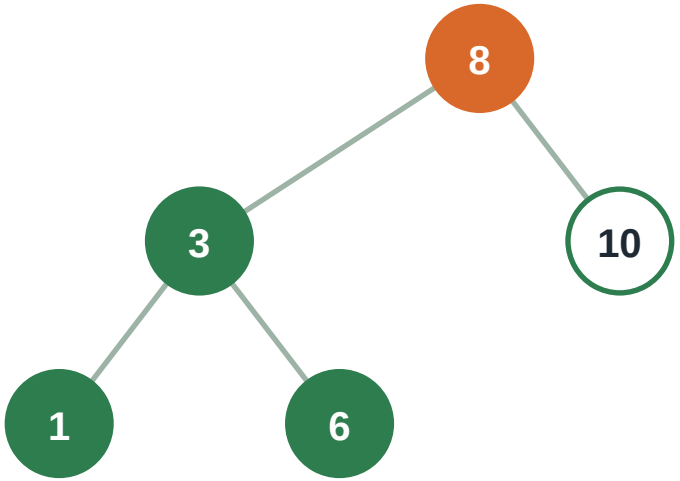
`traverse(8)` → `traverse(3)` → `traverse(6)`

Вывод

`1 3 6`

Обход BST · шаг 7 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Всё поддерево 3 обошли, вернулись в 8. Печатаем 8.

Стек вызовов

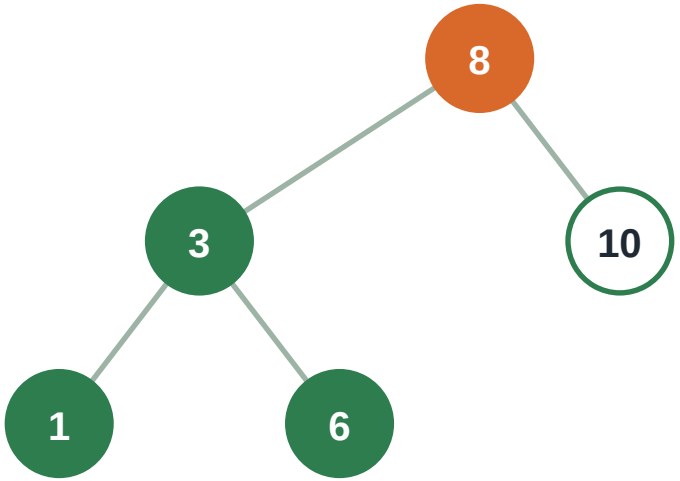
`traverse(8)`

Вывод

`1 3 6 8`

Обход BST · шаг 8 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Осталось правое поддерево корня.

Стек вызовов

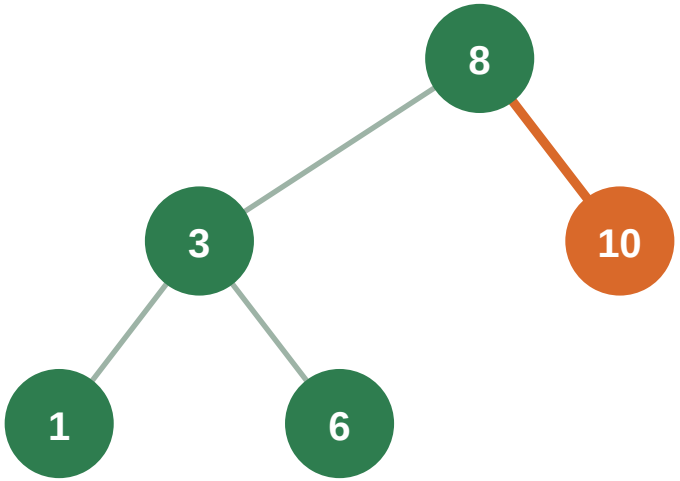
`traverse(8)`

Вывод

`1 3 6 8`

Обход BST · шаг 9 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



10 — лист. Печатаем 10.

Стек вызовов

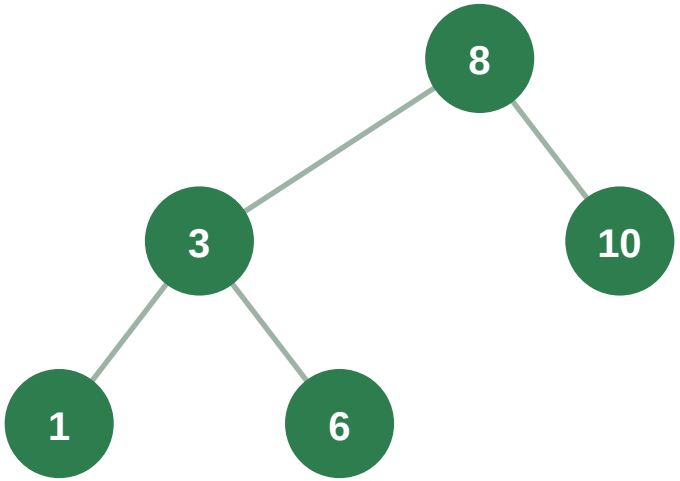
`traverse(8) → traverse(10)`

Вывод

`1 3 6 8 10`

Обход BST · шаг 10 из 10

```
1 void traverse(Node* root) {
2     if (root == nullptr) return;
3     traverse(root->left);
4     std::cout << root->key << ' ';
5     traverse(root->right);
6 }
```



Все вызовы закончились. Ключи напечатаны по возрастанию.

Стек вызовов

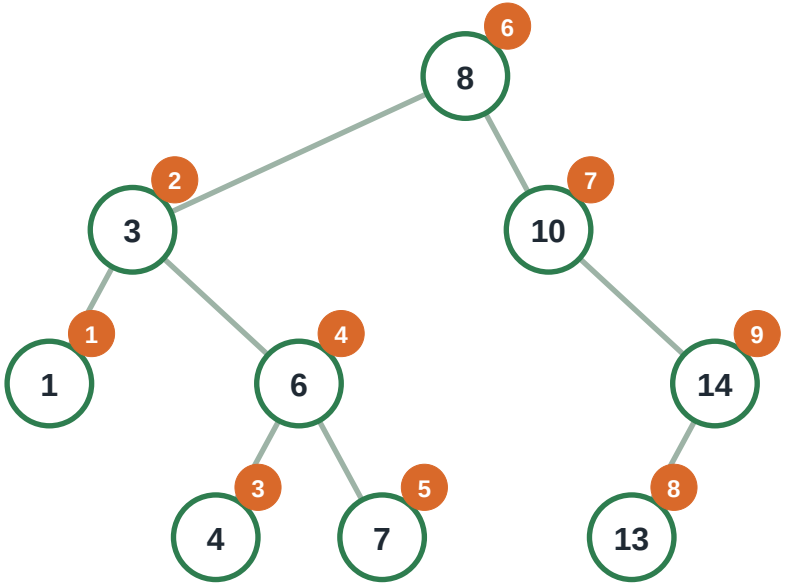
пусто

Вывод

1 3 6 8 10

Итог: обход BST печатает ключи по возрастанию

```
void traverse(Node* root) {
    if (root == nullptr) return;
    traverse(root->left);
    std::cout << root->key << ' ';
    traverse(root->right);
}
```



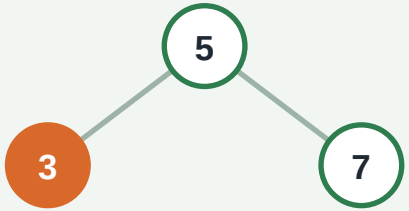
То же самое на большом BST. Рекурсия сама помнит, куда вернуться после левого поддерева.

Порядок: левое поддерево → узел → правое. Оранжевые цифры — очередь печати.

Вывод: `1 3 4 6 7 8 10 13 14`

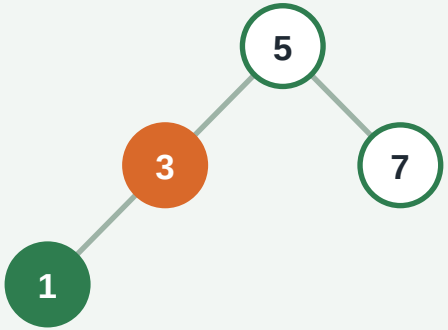
Удаление: три случая

1. Лист



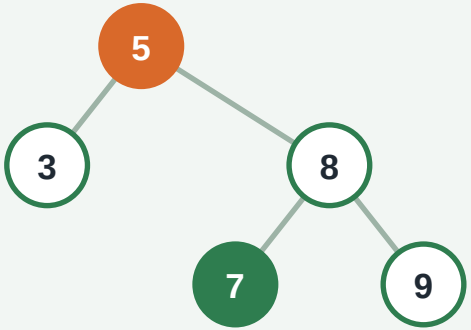
Нет детей. Просто удаляем узел.

2. Один ребёнок



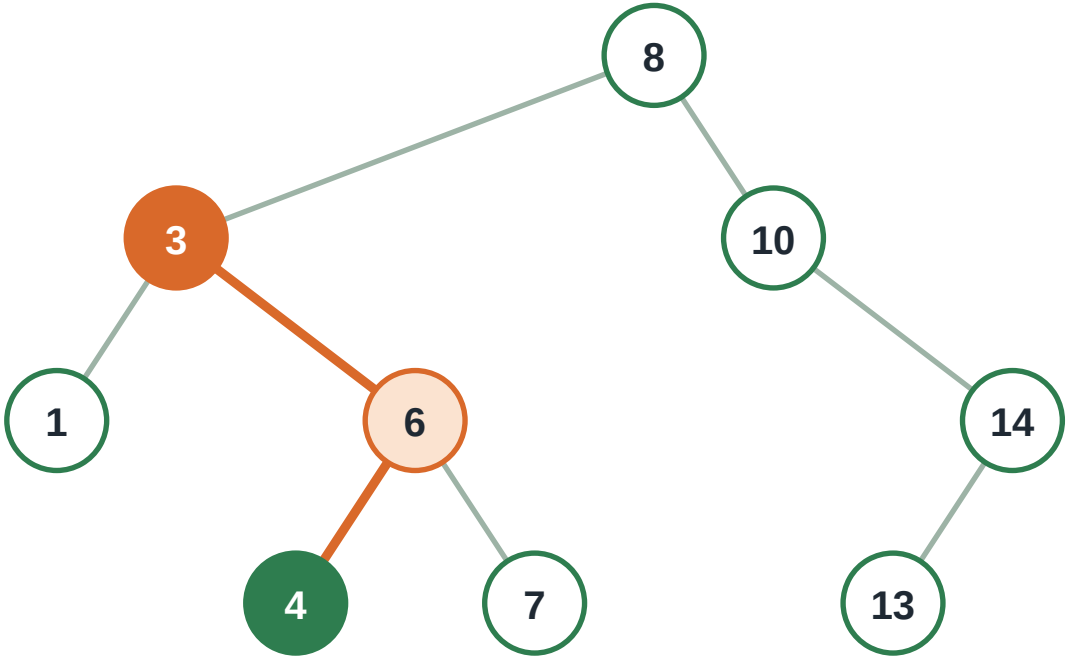
Ребёнок встаёт на место удалённого узла.

3. Два ребёнка



Берём самый маленький узел справа, копируем его ключ и удаляем уже его.

Удаление 3 · шаг 1 из 2

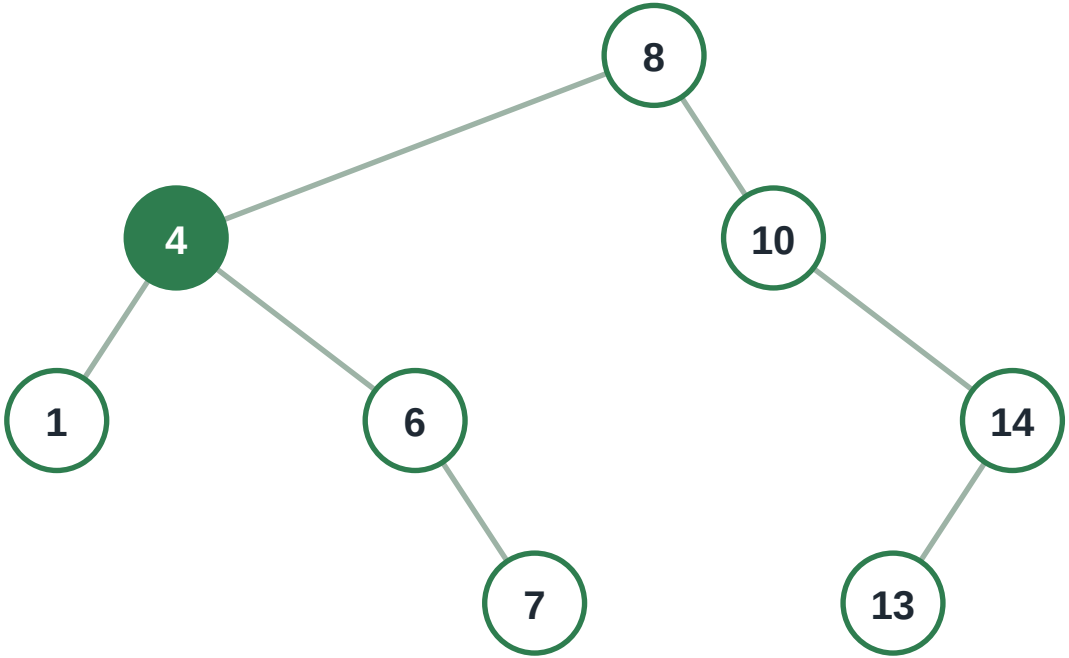


Ищем замену

У 3 два ребёнка. Идём направо (6), потом до упора налево. Самый маленький справа — 4.

Путь: 3 → 6 → 4

Удаление 3 · шаг 2 из 2



Готово!

Копируем 4 на место 3 и удаляем старый узел 4 (он лист — случай 1). Слева от 4 всё меньше, справа всё больше.

Удаление в коде

```
Node* erase(Node* root, int key) {
    if (root == nullptr) return nullptr;
    if (key < root->key) {
        root->left = erase(root->left, key);
    } else if (key > root->key) {
        root->right = erase(root->right, key);
    } else if (root->left == nullptr) {
        Node* child = root->right;
        delete root;
        return child;
    } else if (root->right == nullptr) {
        Node* child = root->left;
        delete root;
        return child;
    } else {
        Node* next = root->right;
        while (next->left != nullptr)
            next = next->left;
        root->key = next->key;
        root->right = erase(root->right, next->key);
    }
    return root;
}
```

1. **Ищем узел**
   Как в find: идём влево или вправо.

2. **Случаи 1 и 2**
   Нет левого или правого ребёнка — возвращаем второго (или nullptr).

3. **Случай 3**
   Минимум справа: шаг направо, потом до упора налево.

delete освобождает память узла. Без него программа бы «теряла» память.

Собираем всё вместе

```
int main() {
    Node* root = nullptr;
    for (int k : {8, 3, 10, 1, 6, 14, 4, 7, 13})
        root = insert(root, k);

    traverse(root);
    std::cout << '\n';

    if (find(root, 6) != nullptr)
        std::cout << "6 found\n";

    root = erase(root, 3);
    traverse(root);

    clear(root);
}
```

Вывод программы

```
1 3 4 6 7 8 10 13 14
6 found
1 4 6 7 8 10 13 14
```

`clear(root)` в конце удаляет все узлы: сначала детей, потом родителя. Код clear устроен как обход.

Часть 4

Проверь себя

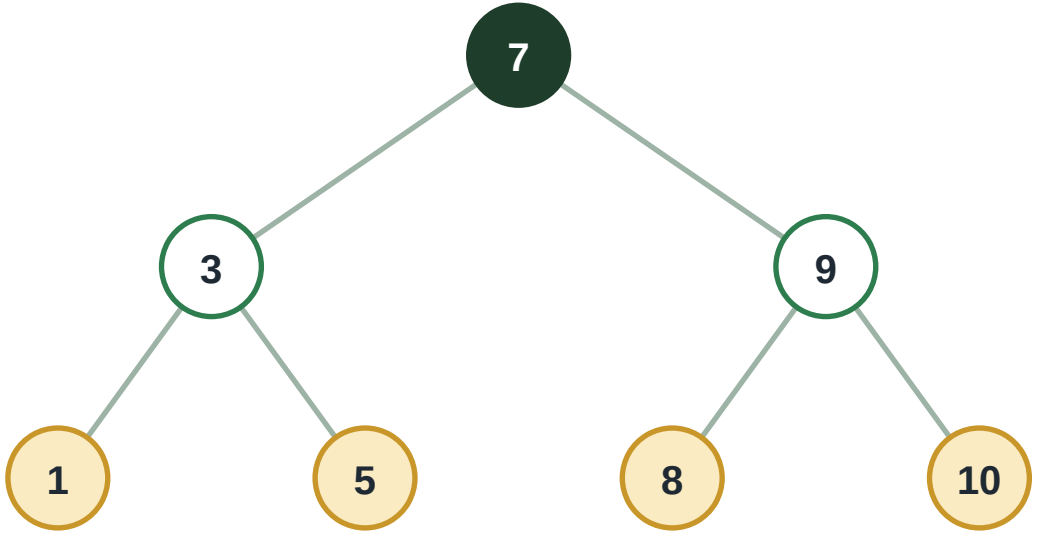
Сначала реши на бумаге, ответы на следующем слайде

Задание 1: построй BST

Вставь по очереди в пустое BST: `7, 3, 9, 1, 5, 8, 10`

1. Нарисуй получившееся дерево.
2. Какая у него высота?
3. Какие узлы — листья?
4. Что выведет traverse?

Ответ к заданию 1



Высота: 2

7 → 3 → 1 — два ребра.

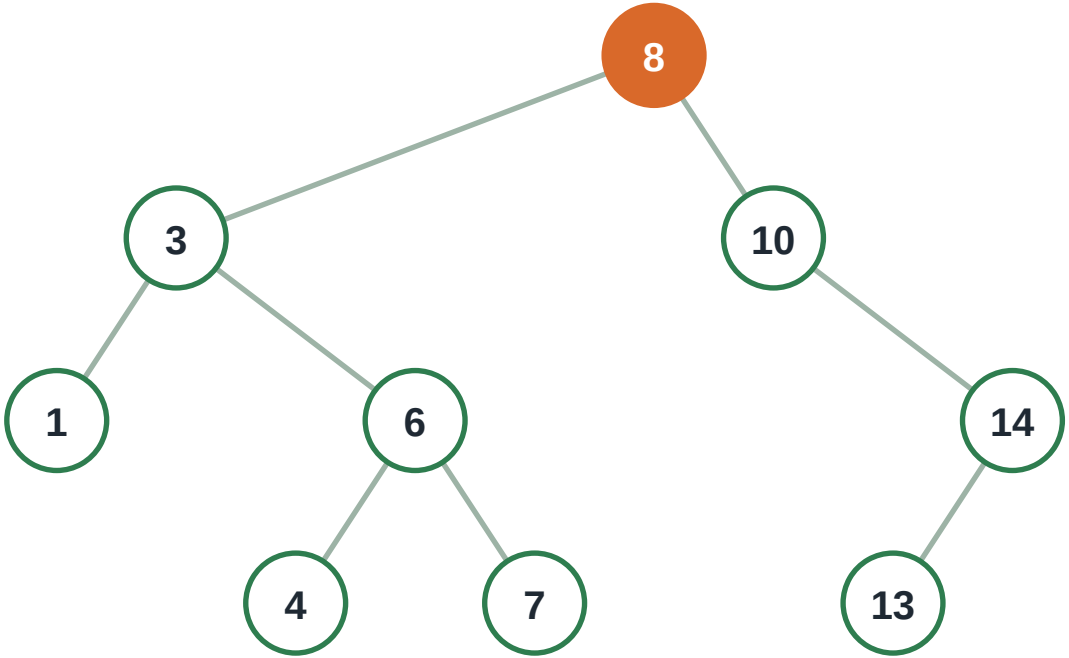
Листья: 1, 5, 8, 10

У них нет детей.

traverse:

`1 3 5 7 8 9 10`

Задание 2: удали корень из BST

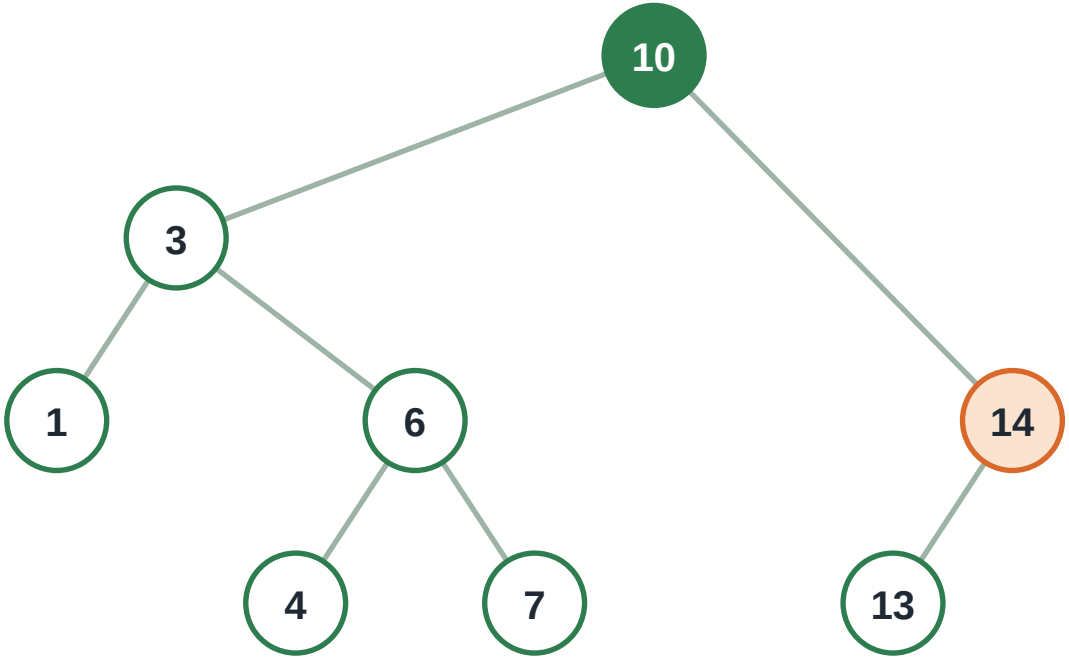


Что будет после `erase(root, 8)`?

Подсказка: у 8 два ребёнка. Какой узел встанет на его место?

Нарисуй новое дерево.

Ответ к заданию 2



Самый маленький справа от 8 — это **10** (у него нет левого ребёнка).

10 переезжает в корень, а его правый ребёнок 14 цепляется к корню справа.

traversе: `1 3 4 6 7 10 13 14`

Посмотри дома: деревья в анимации



«Как работают деревья | Структуры данных»

youtu.be/0BUX_PotA4c

Домашнее задание

Построй дерево для 50, 30, 70, 20, 40, 60, 80, найди в нём 60 и удали 30. Каждый шаг рисуй отдельно, как на слайдах урока.

Часть 5

Практика

Девять задач от простых к сложным: сначала повторяем код урока, потом решаем новое

Шаг 1. Повтори код урока на LeetCode

1 LeetCode 700

Easy

Search in a Binary Search Tree

Дан корень BST и число. Верни узел с этим числом или nullptr, если его нет.

Это наш find()

2 LeetCode 701

Medium

Insert into a Binary Search Tree

Вставь новое число в BST так, чтобы правило «слева меньше, справа больше» сохранилось. Верни корень.

Это наш insert()

3 LeetCode 450

Medium

Delete Node in a BST

Удали узел с заданным ключом и верни корень. Не забудь все три случая: лист, один ребёнок, два ребёнка.

Это наш erase()

В LeetCode узел называется TreeNode, а ключ лежит в поле val. Ссылки в названиях задач кликабельны.

Шаг 2. Codewars: обходим всё дерево

4 Codewars

7 kyu

Binary Tree Search (not BST)

Проверь, есть ли число в дереве: true или false. Дерево не упорядочено, поэтому ищем и слева, и справа.

Как find(), но без правила BST

5 Codewars

6 kyu

Sum The Tree

Найди сумму значений всех узлов дерева, включая корень. Пустой ребёнок — nullptr.

Как traverse(), только вместо cout складываем

6 Codewars

6 kyu

Fun with trees: max sum

Найди наибольшую сумму на пути от корня до листа. Пустое дерево — 0. Путь должен заканчиваться именно в листе.

Ловушка: узел с одним ребёнком — не лист

Здесь дерево обычное, не BST: правило «налево или направо» больше не помогает.

Шаг 3. Codewars: форма дерева

7 Codewars

5 kyu

Fun with trees: array to tree

Построй дерево из массива: заполняй уровни сверху вниз, каждый уровень слева направо. Верни корень.

Узлы создаём через new, как в insert()

8 Codewars

5 kyu

Fun with trees: is perfect

Проверь, идеальное ли дерево: у всех внутренних узлов два ребёнка, и все листья на одной глубине.

Пригодится высота дерева

9 Codewars

5 kyu

Complete Binary Tree

Дан массив — обход слева направо полного двоичного дерева. Восстанови дерево и выведи значения по уровням.

Обход наоборот: от вывода к дереву

Задачи 7–9 сложнее: они про то, как дерево выглядит целиком, а не про один путь.

Главное за урок

1. **Дерево:** граф без циклов: N вершин, N − 1 ребро, один путь между любыми двумя.

2. **Дерево поиска:** слева меньше, справа больше. Поиск отбрасывает половину на каждом шаге.

3. **Баланс:** без него дерево вырождается в «палку». АВЛ и красно-чёрные деревья следят за ним сами.

4. **В C++:** Node с left и right, рекурсивные insert, find, erase и обязательный delete.